

Szerződések funkciók beállítása

Szerződések kezelése

A szerződések kezelése funkció fejlesztése során elkészítettem a szerződések nyilvántartására szolgáló felhasználói felületet. Az űrlapon lehetővé tettem új szerződések létrehozását, ahol kiválasztható az ingatlan és a bérlő, valamint megadhatók a szerződéshez tartozó adatok. A rendszer automatikusan betölti az adatbázisban található ingatlanokat és bérlőket, így a felhasználónak nem kell manuálisan megadnia az azonosítókat. Megvalósítottam a szerződések módosításának lehetőségét is, amely során a kiválasztott szerződés adatai automatikusan betöltődnek az űrlapba szerkesztés céljából. A funkció tesztelése során ellenőriztem az új szerződések létrehozását, a meglévő adatok módosítását, valamint a szerződések törlését is. A rendszerben kialakítottam a szerződések státusz szerinti szűrésének lehetőségét. A felhasználó kiválaszthatja, hogy az összes, az aktív vagy a lejárt szerződéseket szeretné megjeleníteni. A kiválasztott szűrő alapján a frontend dinamikusan szűri a megjelenített adatokat. Ezzel jelentősen egyszerűbbé vált a nagyobb mennyiségű szerződés áttekintése. A funkció működését különböző státuszú tesztadatokkal ellenőriztem.

```
import { useEffect, useState } from "react";
import "../styles/szerzodesek.css";

function Szerzodesek() {

  const [szerzodesek, setSzerzodesek] = useState([]);
  const [berlok, setBerlok] = useState([]);
  const [ingatlanok, setIngatlanok] = useState([]);
  const [szerkesztettId, setSzerkesztettId] = useState(null);
  const [statuszSzuro, setStatuszSzuro] = useState("Mind");

  const [ujSzerzodes, setUjSzerzodes] = useState({
    ingatlanid: "",
    berloid: "",
    kezdetdatum: "",
    vegdatum: "",
    havidij: "",
    statusz: "Aktív"
  });

  const szerzodesekBetoltese = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/szerzodeslistazas"
      );

      const adatok = await valasz.json();

      setSzerzodesek(adatok);

    }

  }

}
```

```
    } catch (err) {  
  
        console.error(err);  
  
    }  
};  
  
const berlokBetoltese = async () => {  
  
    try {  
  
        const valasz = await fetch(  
            "http://localhost:3001/api/berlolistazas"  
        );  
  
        const adatok = await valasz.json();  
  
        setBerlok(adatok);  
  
    } catch (err) {  
  
        console.error(err);  
  
    }  
};  
  
const ingatlanokBetoltese = async () => {  
  
    try {  
  
        const valasz = await fetch(  
            "http://localhost:3001/api/ingatlanlistazas"  
        );  
  
        const adatok = await valasz.json();  
  
        setIngatlanok(adatok);  
  
    } catch (err) {  
  
        console.error(err);  
  
    }  
};
```

```
useEffect(() => {

    szerzodesekBetoltese();
    berlokBetoltese();
    ingatlanokBetoltese();

}, []);

const mentes = async (e) => {

    e.preventDefault();

    try {

        let url =
            "http://localhost:3001/api/szerzodesletrehozas";

        let method = "POST";

        if (szerkesztettId) {

            url =
                `http://localhost:3001/api/szerzodesmodositas/${szerkesztettId}`;

            method = "PUT";

        }

        const valasz = await fetch(
            url,
            {
                method,
                headers: {
                    "Content-Type":
                        "application/json"
                },
                body: JSON.stringify(
                    ujSzerzodes
                )
            }
        );

        const adat =
            await valasz.json();

        alert(adat.uzenet);

        setUjSzerzodes({
```

```
        ingatlanid: "",
        berloid: "",
        kezdetdatum: "",
        vegdatum: "",
        havidij: "",
        statusz: "Aktív"
    });

    setSzerkesztettId(null);

    szerzodesekBetoltese();

} catch (err) {

    console.error(err);

    alert("Hiba történt!");

}

};

const torles = async (id) => {

    if (
        !window.confirm(
            "Biztosan töröld a szerződést?"
        )
    ) {
        return;
    }

    try {

        const valasz = await fetch(
            `http://localhost:3001/api/szerzodestorles/${id}`,
            {
                method: "DELETE"
            }
        );

        const adat =
            await valasz.json();

        alert(adat.uzenet);

        szerzodesekBetoltese();
```



```
        {
            ingatlan.cim
        }
    </option>

    )
}}

</select>

<select
    value={
        ujSzerzodes.berloid
    }
    onChange={(e) =>
        setUjSzerzodes({
            ...ujSzerzodes,
            berloid:
                e.target.value
        })
    }
    required
>

    <option value="">
        Válassz bérlőt
    </option>

    {berlok.map(
        (berlo) => (

            <option
                key={
                    berlo.id
                }
                value={
                    berlo.id
                }
            >
                {
                    berlo.nev
                }
            </option>

        )
    )}

</select>
```

```
<input
  type="date"
  value={
    ujSzerzodes.kezdetdatum
  }
  onChange={(e) =>
    setUjSzerzodes({
      ...ujSzerzodes,
      kezdetdatum:
        e.target.value
    })
  }
  required
/>
```

```
<input
  type="date"
  value={
    ujSzerzodes.vegdatum
  }
  onChange={(e) =>
    setUjSzerzodes({
      ...ujSzerzodes,
      vegdatum:
        e.target.value
    })
  }
  required
/>
```

```
<input
  type="number"
  placeholder="Havi díj"
  value={
    ujSzerzodes.havidij
  }
  onChange={(e) =>
    setUjSzerzodes({
      ...ujSzerzodes,
      havidij:
        e.target.value
    })
  }
  required
/>
```

```
<select
```

```
        value={
          ujSzerzodes.statusz
        }
        onChange={(e) =>
          setUjSzerzodes({
            ...ujSzerzodes,
            statusz:
              e.target.value
          })
        }
      >

      <option value="Aktív">
        Aktív
      </option>

      <option value="Lejárt">
        Lejárt
      </option>

    </select>

    <button
      type="submit"
    >
      {
        szerkesztettId
          ? "Módosítás mentése"
          : "Szerződés létrehozása"
      }
    </button>

  </form>

  <div className="filter-container">

    <select
      value={statuszSzuro}
      onChange={(e) =>
        setStatuszSzuro(
          e.target.value
        )
      }
    >

      <option value="Mind">
        Összes szerződés
      </option>
```



```
        <option value="Aktív">
            Aktív
        </option>

        <option value="Lejárt">
            Lejárt
        </option>

    </select>

</div>

<div className="tenant-grid">

    {szerzodesek
        .filter((szerzodes) => {

            if (
                statuszSzuro ===
                "Mind"
            ) {
                return true;
            }

            return (
                szerzodes.statusz ===
                statuszSzuro
            );

        })
        .map(
            (szerzodes) => (

                <div
                    className="tenant-card"
                    key={
                        szerzodes.id
                    }
                >

                    <h3>
                        {
                            szerzodes.ingatlancim
                        }
                    </h3>

                    <p>
```

```
<strong>
    Bérlő:
</strong>{" " }
{
    szerzodes.berlonev
}
</p>

<p>
    <strong>
        Kezdet:
    </strong>{" " }
    {
        new Date(
            szerzodes.kezdetdatum
        ).toLocaleDateString(
            "hu-HU"
        )
    }
</p>

<p>
    <strong>
        Vége:
    </strong>{" " }
    {
        new Date(
            szerzodes.vegdatum
        ).toLocaleDateString(
            "hu-HU"
        )
    }
</p>

<p>
    <strong>
        Havi díj:
    </strong>{" " }
    {
        szerzodes.havidij
    } Ft
</p>

<p>

    <strong>
        Státusz:
    </strong>
```

```
<span
  className={
    szerzodes.statusz ===
    "Aktív"
      ? "active-status"
      : "expired-status"
  }
>
  {" "}
  {
    szerzodes.statusz
  }
</span>

</p>

<div className="buttons">

  <button
    type="button"
    className="edit-btn"
    onClick={() => {

      setSzerkesztettId(
        szerzodes.id
      );

      setUjSzerzodes({
        ingatlanid:
          szerzodes.ingatlanid,
        berloid:
          szerzodes.berloid,
        kezdetdatum:
          szerzodes.kezdetdatum?.split("T")[0] ||
          szerzodes.kezdetdatum,
        vegdatum:
          szerzodes.vegdatum?.split("T")[0] ||
          szerzodes.vegdatum,
        havidij:
          szerzodes.havidij,
        statusz:
          szerzodes.statusz
      });

      window.scrollTo({
        top: 0,
        behavior: "smooth"
```

```
        });

        }}

        >
        Módosítás
    </button>

    <button
        type="button"
        className="delete-btn"
        onClick={() =>
            torles(
                szerzodes.id
            )
        }
    >
        Törlés
    </button>

</div>

</div>

    )
    })

</div>

</div>

);

}

export default Szerzodesek;
```

Szerződések oldal stíluslapja

A szerződések oldal megjelenítéséhez külön CSS stíluslapot készítettem. A célom egy letisztult, könnyen áttekinthető és felhasználóbarát felület kialakítása volt. A szerződések kártyás elrendezésben jelennek meg, ami megkönnyíti az adatok áttekintését. Külön stílusokat alkalmaztam az űrlapokra, a gombokra és a státuszjelölésekre is. A reszponzív kialakításnak köszönhetően az oldal kisebb képernyőkön és mobil eszközökön is megfelelően használható.

```
.tenant-container {
    max-width: 1400px;
    margin: 30px auto;
    padding: 20px;
```

```
}

.tenant-container h1 {
  text-align: center;
  margin-bottom: 30px;
  font-size: 42px;
}

.tenant-form {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 3px 10px rgba(0,0,0,0.15);
  margin-bottom: 30px;
}

.tenant-form select,
.tenant-form input {
  flex: 1;
  min-width: 180px;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 8px;
}

.tenant-form button {
  background: #2e7d32;
  color: white;
  border: none;
  padding: 12px 25px;
  border-radius: 8px;
  cursor: pointer;
}

.tenant-form button:hover {
  background: #1b5e20;
}

.tenant-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill,minmax(320px,1fr));
  gap: 20px;
}

.tenant-card {
```

```
    background: white;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 3px 10px rgba(0,0,0,0.15);
}

.tenant-card h3 {
    margin-bottom: 15px;
    color: #2e7d32;
}

.tenant-card p {
    margin: 8px 0;
}

.buttons {
    margin-top: 15px;
    display: flex;
    gap: 10px;
}

.edit-btn {
    background: #1976d2;
    color: white;
    border: none;
    padding: 10px 15px;
    border-radius: 8px;
    cursor: pointer;
}

.edit-btn:hover {
    background: #0d47a1;
}

.delete-btn {
    background: #d32f2f;
    color: white;
    border: none;
    padding: 10px 15px;
    border-radius: 8px;
    cursor: pointer;
}

.delete-btn:hover {
    background: #b71c1c;
}

.filter-container {
```

```
    margin: 20px 0;
}

.filter-container select {
    padding: 10px;
    border-radius: 8px;
}

.active-status {
    color: green;
    font-weight: bold;
}

.expired-status {
    color: red;
    font-weight: bold;
}
```

Szerződés létrehozása

A szerződés létrehozása funkció elkészítése során kialakítottam az adatbeviteli űrlapot, amely lehetővé teszi új szerződések rögzítését. Az űrlapon kiválasztható az ingatlan és a bérlő, valamint megadható a szerződés kezdő és befejező dátuma, a havi díj és a státusz. A megadott adatokat POST kérés segítségével továbbítottam a backend felé. A backend az adatokat ellenőrizte, majd eltárolta a szerződések adatbázistáblájában. A funkció tesztelése során ellenőriztem, hogy az új szerződések sikeresen bekerülnek az adatbázisba.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

    const {
        ingatlanid,
        berloid,
        kezdetdatum,
        vegdatum,
        havidij,
        statusz
    } = req.body;

    let conn;

    try {

        conn = await pool.getConnection();
```

```
    await conn.query(
      `INSERT INTO szerzodesek
      (ingatlanid, berloid, kezdetdatum, vegdatum, havidij, statusz)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [
        ingatlanid,
        berloid,
        kezdetdatum,
        vegdatum,
        havidij,
        statusz
      ]
    );

    res.json({
      siker: true,
      uzenet: "Szerzodes sikeresen létrehozva!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```

Szerződések listázása

A szerződések listázása funkció megvalósításával lehetővé tettem a rendszerben tárolt szerződések megjelenítését. A frontend GET kérés segítségével lekérdezi a backendtől a szerződések adatait. A megjelenített listában látható az ingatlan címe, a bérlő neve, a szerződés időtartama, a havi díj és a státusz. A felhasználói felület áttekinthető kártyás megjelenítést kapott. A funkció működését több tesztadattal is ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;
```



```
    try {

      conn = await pool.getConnection();

      const adatok = await conn.query(`
        SELECT
          szerzodesek.*,
          users.nev AS berlonev,
          ingatlanok.cim AS ingatlancim
        FROM szerzodesek
        INNER JOIN users
          ON users.id = szerzodesek.berloid
        INNER JOIN ingatlanok
          ON ingatlanok.id = szerzodesek.ingatlanid
        ORDER BY szerzodesek.id DESC
      `);

      res.json(adatok);

    } catch (err) {

      console.error(err);

      res.status(500).json(err);

    } finally {

      if (conn) {
        conn.release();
      }

    }

  });

module.exports = router;
```

Szerződés módosítása

A szerződés módosítása funkció elkészítése során lehetőséget biztosítottam a meglévő szerződések adatainak szerkesztésére. A kiválasztott szerződés adatai automatikusan betöltődnek az űrlapba. A felhasználó módosíthatja a kezdő és végdátumot, a havi díjat, valamint a szerződés státuszát. A mentést követően a rendszer PUT kérés segítségével frissíti az adatbázisban tárolt adatokat. A módosítások helyességét sikeres adatbázis-frissítéssel ellenőriztem.

```
const express = require("express");
const router = express.Router();
```

```
const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    kezdetdatum,
    vegdatum,
    havidij,
    statusz
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE szerzodesek
      SET
      kezdetdatum = ?,
      vegdatum = ?,
      havidij = ?,
      statusz = ?
      WHERE id = ?`,
      [
        kezdetdatum,
        vegdatum,
        havidij,
        statusz,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Szerzodes modositva!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});
```

```
    }  
  
  });  
  
module.exports = router;
```

Szerződés törlése

A szerződés törlése funkció megvalósításával lehetővé tettem a már nem szükséges szerződések eltávolítását. A törlés előtt a rendszer megerősítést kér a felhasználótól a véletlen adatvesztés elkerülése érdekében. Jóváhagyás esetén DELETE kérés kerül elküldésre a backendnek. A backend eltávolítja a kiválasztott rekordot az adatbázisból. A funkció tesztelése során ellenőriztem, hogy a törölt szerződések a listából és az adatbázisból is megfelelően eltűnnek.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.delete("/:id", async (req, res) => {  
  
  let conn;  
  
  try {  
  
    conn = await pool.getConnection();  
  
    await conn.query(  
      "DELETE FROM szerzodesek WHERE id = ?",  
      [req.params.id]  
    );  
  
    res.json({  
      siker: true,  
      uzenet: "Szerzodes torolve!"  
    });  
  
  } catch (err) {  
  
    res.status(500).json(err);  
  
  } finally {  
  
    if(conn) conn.release();  
  
  }  
  
});
```

```
module.exports = router;
```

The screenshot shows a web browser at localhost:3000/szerzodesek. The page title is 'Szerződések'. Below the title is a form for creating a new contract, labeled 'Szerződés létrehozása'. The form contains several input fields: 'Válassz ingatlant' (dropdown), 'Válassz bérletet' (dropdown), two date pickers (formatted as 'éééé. hh. nn.'), 'Havi díj' (text input), and 'Aktív' (dropdown). Below the form is a button labeled 'Szerződés létrehozása'. Below the button is a dropdown menu labeled 'Összes szerződés'. Below the dropdown menu is a card displaying details for a contract: 'Debrecen, Fő utca 21.', 'Bérlő: Papp Renáta', 'Kezdet: 2026. 01. 01.', 'Vége: 2026. 12. 31.', 'Havi díj: 120000.00 Ft', and 'Státusz: Aktív'. At the bottom of the card are two buttons: 'Módosítás' (blue) and 'Törölés' (red).

1.Kép: Szerződések HTML oldal kimenete.